

DAY 4 · LECTURE & DEMO · 90 MINUTES

The Admiral's Arsenal

Cloud-Native Classrooms: vCluster, KubeVirt & Chaos

Three days of building. This morning, the most advanced manoeuvres in the fleet.

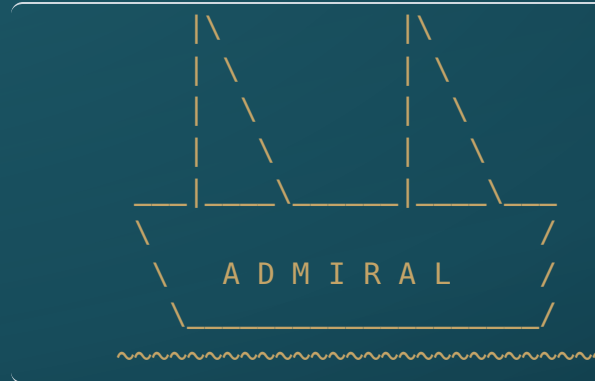


Where we are

- ♦ **Days 1–3:** you built, connected, templated, and automated — container images, live Pods, three-tier deployments, GitOps pipelines.
- ♦ This morning, Admiral Bash opens the arsenal: the tools that make *this whole curriculum portable to your own college*.
- ♦ Four parts — three tools, one bridge to the lab:
 - **Part I** — The Bunk and the Ship: `vCluster`
 - **Part II** — The Legacy Fleet: `KubeVirt`
 - **Part III** — Summoning the Storm: Chaos Engineering
 - **Part IV** — The Ultimate Exam (+ what comes next)
- ♦ Then the break — and the Pirate strikes.

PART I

The Bunk and the Ship



"What a ship is... is freedom." — Captain Jack Sparrow, Pirates of the Caribbean: The Curse of the Black Pearl

The problem with namespaces

- ♦ So far, each student gets their own **namespace** — their own patch of the shared ocean.
- ♦ A namespace is like a **bunk on a ship**: private, but the ship's rules still govern everything.
- ♦ Students **cannot**:
 - install Custom Resource Definitions (CRDs)
 - create cluster-level resources
 - touch anything outside their namespace
- ♦ For most labs, that is fine. For advanced courses, it is a cage.

The other extreme: cluster-admin

- ♦ Give a student `cluster-admin` on the shared cluster and they have the keys to the whole ship.
- ♦ One misfired `kubectl delete` wipes resources belonging to 29 other students.
- ♦ One runaway process drains the cluster's CPU for the whole room.
- ♦ One mistake during a CRD exercise breaks the control plane for everyone.

Sharing `cluster-admin` on a live teaching cluster is not a risk — it is a when, not an if.

The bunk is too small. The ship is too dangerous.

NAMESPACE (bunk)	SHARED CLUSTER (ship)
+-----+	+-----+
- no CRDs	+ full cluster-admin
- no admin	- one student sinks all
- restricted	- no isolation at all
+-----+	+-----+

You need something in between.

- ♦ Students need the *feeling* of cluster-admin without the blast radius.
- ♦ The answer: give each student their **own Kubernetes cluster** — without buying 30 clusters.

vCluster: a cluster inside a namespace

- ♦ **vCluster** deploys a full, isolated Kubernetes API server *inside* a single namespace on the host cluster.
- ♦ From the student's perspective: a real multi-node cluster, full `cluster-admin`, CRDs welcome.
- ♦ From the host cluster's perspective: one namespace, one set of Pods — ordinary traffic.
- ♦ The host cluster never sees the student's internal objects; the student never touches the host.

```
HOST CLUSTER
+-----+
| namespace: student-alice |
| +-----+ |
| | vCluster API server (a Pod) | |
| | student sees: nodes, CRDs, admin | |
| +-----+ |
+-----+
```

vCluster: what the instructor controls

- ♦ **Spin up:** one Helm install per student — or scripted for 30 at once.
- ♦ **Tear down:** `helm uninstall` — the whole cluster disappears with it.
- ♦ **Isolation:** a student cannot escape their vCluster into the host.
- ♦ **Resource limits:** the namespace wrapping the vCluster can have CPU/memory quotas.
- ♦ Students get the *experience* of owning a cluster; the instructor keeps the keys to the ship.

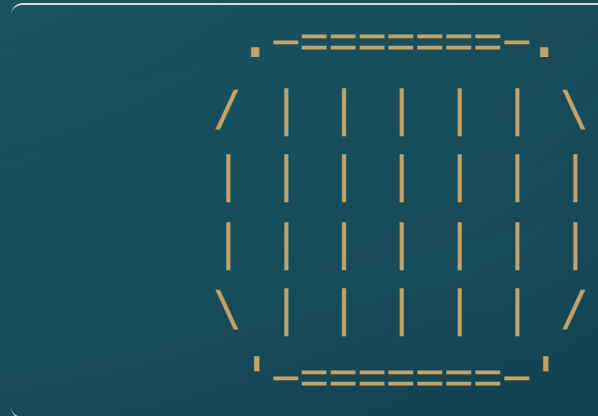
vCluster: the classroom payoff

- ♦ **30 students, 30 real clusters** — on one shared host, no provisioning from IT.
- ♦ Students can install operators, experiment with CRDs, break things freely.
- ♦ Advanced courses — service mesh, multi-tenant SaaS, operator development — all become teachable without a cloud bill per student.
- ♦ When a student melts their cluster: `helm uninstall` + `helm install` — reset in under a minute.

This is what it looks like when IT is no longer the bottleneck on what you can teach.

PART II

The Legacy Fleet



"Don't scuttle the old fleet just because ye launched a new one. Some cargoes need the old hull." — the Boatswain

The hesitation every faculty member has

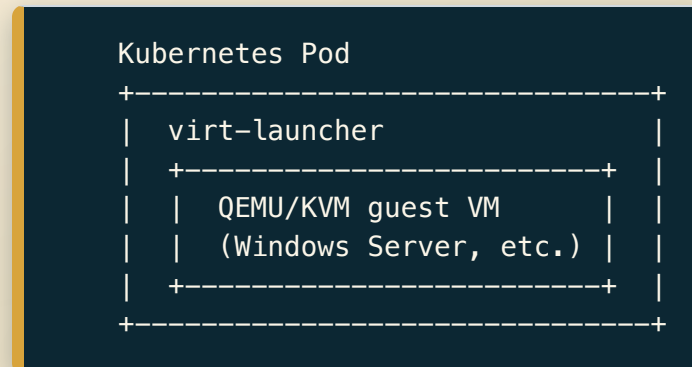
- ♦ "This is great — but I still have to teach **Windows Server**."
- ♦ "My program covers **Active Directory**, DHCP, legacy Linux admin."
- ♦ "I can't abandon that curriculum just because Kubernetes is shiny."

This is a legitimate structural constraint, not a failure of imagination.

A Kubernetes-only lab environment looks like a step backward if you still owe your students Windows Server administration.

KubeVirt: VMs as Kubernetes objects

- ♦ **KubeVirt** is a CNCF project that adds a new resource type to Kubernetes: the `VirtualMachine`.
- ♦ Under the hood, it wraps a standard **QEMU/KVM** virtual machine inside a Kubernetes Pod.
- ♦ The VM boots like any desktop VM — Windows, Linux, whatever image you supply.
- ♦ From Kubernetes' point of view: just another Pod to schedule, watch, and restart.



KubeVirt: demo — a VM on the cluster

Live demo — instructor projects terminal.

- ♦ Apply a `VirtualMachine` manifest to the cluster.
- ♦ Start it: `virtctl start my-windows-vm`
- ♦ Console in: `virtctl console my-windows-vm`

The VM is live on the **same virtual network** as the Day 2 web Pods.

```
ping from windows-vm --> frontend-pod    (it works)
curl from backend-pod --> windows-vm     (it works)
```

One cluster. Containers and VMs sharing the same network fabric.

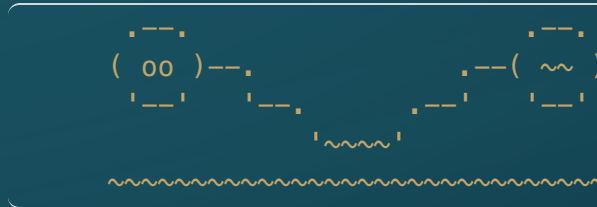
KubeVirt: the classroom payoff

- ♦ Teach Windows Server administration and containerized microservices in the **same lab session**.
- ♦ Students configure Active Directory — then connect it to a containerized app — in one unified environment.
- ♦ No separate VM infrastructure. No separate network. No separate IT queue.
- ♦ Hardcore IT programs use this to move fully onto Kubernetes **without throwing away** a decade of legacy lab content.

Legacy curriculum	Kubernetes curriculum
Windows Server	containerized apps
Active Directory	Kubernetes RBAC
DHCP / DNS	CoreDNS + Services
All on the same cluster	← same cluster

PART III

Summoning the Storm



"A man can be destroyed but not defeated." — Ernest Hemingway, The Old Man and the Sea

Real systems fail

- ♦ No production system runs forever without a fault.
- ♦ The question is not *whether* it will fail — it is *whether you find out* from your own monitoring, or from a flood of angry users.
- ♦ **Chaos Engineering** means intentionally breaking your own system — under controlled conditions, in your own lab — to *prove* it can survive.
- ♦ This is not recklessness. It is the methodology of modern **Site Reliability Engineering (SRE)**.

"Hope is not a reliability strategy." — SRE axiom

The tools: Chaos Mesh and LitmusChaos

- ♦ **Chaos Mesh** — CNCF-hosted. Chaos experiments are Kubernetes CRDs: apply a manifest, the chaos begins; delete it, the storm stops.
- ♦ **LitmusChaos** — CNCF-hosted. Workflow-oriented chaos with built-in dashboards and test reports.
- ♦ Both run *inside* the cluster — no external attack infrastructure needed.
- ♦ Both define faults as ordinary Kubernetes objects — inspectable, version-controlled, shareable.

*For this seminar we use **Chaos Mesh**. The experiments you will see this morning are already staged.*

The kinds of faults

PodChaos	NetworkChaos	StressChaos
-----	-----	-----
pod-kill	loss (%)	CPU hog
pod-failure	delay (ms)	memory hog
container-	bandwidth	
kill	limit	

- ♦ **PodChaos** — violently kill a Pod (or container) on a schedule.
- ♦ **NetworkChaos** — drop a percentage of packets, inject latency, cap bandwidth — *between specific namespaces*.
- ♦ **StressChaos** — flood CPU or memory inside a running container.

All are scoped with a **selector**: target by namespace, label, or Pod name.

What a chaos manifest looks like

```
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
  name: kraken-packet-loss
  namespace: chaos
spec:
  action: loss
  mode: all
  duration: "90m"
  loss:
    loss: "40"
  selector:
    namespaces: ["blackbeard", "annebonny"]
```

- ♦ One manifest. Drops **40%** of packets in both directions for 90 minutes.
- ♦ Apply it: the storm begins. Delete it: instant calm.
- ♦ Students can see this object — that visibility is core to the exercise.

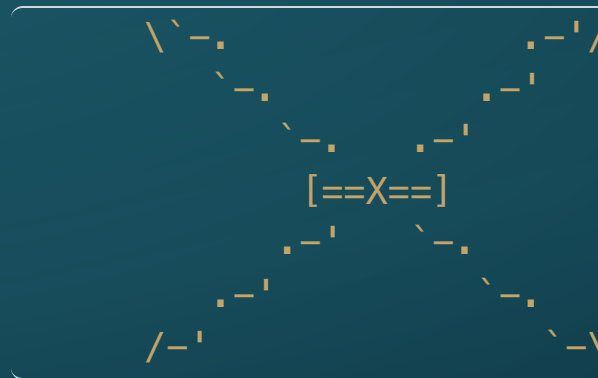
Chaos Engineering as SRE practice

- ♦ **Netflix Simian Army** coined the practice — randomly terminate production instances to prove resilience.
- ♦ **Google SRE** runs "DiRT" (Disaster Recovery Training) — planned, scoped chaos against live services.
- ♦ The insight: a system that is never tested under failure is a system whose failure mode is unknown.
- ♦ **Your curriculum already has the scaffolding**: Deployments auto-respawn Pods; ArgoCD auto-heals GitOps drift. The students built a resilient system *without knowing it was resilient*.

After the break, they will find out.

PART IV

The Ultimate Exam



"A smooth sea never made a skilled sailor." — a sailor's proverb

Chaos as assessment

- ♦ A traditional exam asks: *"Do you know what a NetworkPolicy is?"*
- ♦ A chaos exam asks: *"Fix the live system. It is losing 40% of its packets. Go."*
- ♦ The `NetworkChaos` manifest **is** the test paper.
- ♦ The student's `kubectl get events` output **is** the answer sheet.
- ♦ You grade the **recovery**, not the recall.

This is the most authentic assessment a DevOps / systems administration program can run.

What makes it gradeable

What you observe	What it proves
<code>kubectl get events</code> before panicking	systematic diagnosis
Finding the <code>NetworkChaos</code> object	connecting symptoms to cause
Writing a <code>NetworkPolicy</code> via GitOps	disciplined response under pressure
60 s of clean traffic while attack runs	actual remediation

- ♦ No rubric guessing. No "partial credit for knowing the command."
- ♦ The system either serves traffic reliably or it does not.

The bridge: what this morning built toward

This morning's three topics are not independent demos. They form a stack:

```
vCluster    -> safe, full clusters for every student  
KubeVirt    -> legacy curriculum lives on the same platform  
Chaos Mesh  -> authentic, self-grading assessment built in
```

- ♦ Together they are the answer to: *"How do I run this curriculum at my college?"*
- ♦ After the break, you will not just understand Chaos Mesh — you will be inside a live attack.

Foreshadow: *during the 10:30 break, the Pirate boards the cluster. When you return, something will already be wrong. That is not a drill.*

Instructor Superpower

THE CLOUD-NATIVE CLASSROOM STACK

This morning's three tools all point the same way — at what you can now build yourself:

- ♦ **vCluster** — 30 students, 30 real clusters on one shared host. Stand it up yourself.
- ♦ **KubeVirt** — teach Windows Server and legacy Linux admin on the same unified cluster as your container labs.
- ♦ **Chaos Mesh** — turn a `NetworkChaos` manifest into an auto-gradeable capstone exam.
- ♦ The environments you can choose to support have radically expanded. You don't wait for IT to say "sure, we can support that" — you build it, for a lesson, a course, or a seminar like this one.

Up next: Lab 01 — The Pirate Strikes

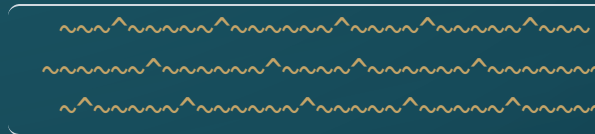
- ♦ The cluster is about to come **under attack**.
- ♦ Pods in your namespace will die and respawn. Your 3-tier pipeline will flicker.
- ♦ A Pirate has boarded — running live **Chaos Mesh** experiments against the fleet.
- ♦ Your mission: **trace the root cause and stabilize your logistics pipeline** while the attack is still running.

What you will use:

- ♦ `k9s` — watch the carnage in real time
- ♦ `kubectl get events` — read the ship's incident log
- ♦ `kubectl get podchaos, networkchaos -A` — name your enemy
- ♦ `NetworkPolicy` via **GitOps** — raise the blockade

After the break, the storm is already running. Do not panic — observe first.

To the storm.



The arsenal is loaded. The Pirate is coming. Prove the fleet is ready.

